

Сессионное задание. Настройка доступа к веб-приложению через VPN и интеграция авторизации в веб-приложение

Contents

Постановка задачи	2
Введение (цель работы)	2
Настройка работы VPN.....	2
Описание стенда VPN.....	2
Описание архитектуры VPN.....	3
Скриншоты установки и настройки VPN	3
Настройка WireGuard.....	4
Как устроено веб-приложение.....	8
Общее описание.....	8
Исходный код	8
Архитектура сайта и системы авторизации	10
Используемое программное обеспечение.....	10
Настройка работы в докер	11
Доступ к веб-приложению только через VPN.....	12
Скриншоты веб-приложения	12
Интеграция авторизации в веб-приложение.....	13
Настройка Keycloak.....	14
Настраиваем docker compose.....	14
Настройка свойств Keycloak.....	15
Проверка входа с помощью аутентификации	20
Проверка, что сайт работает только через VPN.....	20
Выводы и сложности.....	22

Список рисунков

Рисунок 1. Архитектура настроенного VPN-решения	3
Рисунок 2. Архитектура стенда для выполнения итогового семестрового задания	10

Постановка задачи

1. Создать веб-приложение
 - а. Создать простое веб-приложение, имеющее больше одной страницы, используя любой технологический стек.
 - б. На страницах могут быть только строки текста, чтобы была возможность отличать их друг от друга.
2. Настроить VPN
 - а. Настроить любой VPN-протокол, сервер можно (но не обязательно) расположить на машине с веб-сервером.
 - б. Обеспечить доступ к веб-приложению с машины-клиента только с включенным VPN.
3. Интегрировать авторизацию в веб-приложение
 - а. Настроить авторизацию в веб-приложении с использованием стандарта OAuth 2.0. Сервером авторизации может выступать Google или любое другое решение.

Допустимо использование любых операционных систем для виртуальной машины.

Введение (цель работы)

Продемонстрировать работу веб-приложения через VPN-канал, а также интегрировать авторизацию в веб-приложение. Все должно работать через VPN-канал.

Настройка работы VPN

Стенд берется из выполненной работы по домашнему заданию N2.

Описание стенда VPN

Ubuntu / будущий WireGuard Server

- интерфейс: ens33
- IP: 192.168.179.129/24

Fedora / будущий WireGuard Client

- интерфейс: ens160
- IP: 192.168.179.130/24

Обе VM находятся в одной сети 192.168.179.0/24

Шлюз у обеих виртуальных машин: 192.168.179.2

Для организации защищённого доступа к веб-приложению используем **WireGuard**. VPN-сервер настраиваем на виртуальной машине Ubuntu с физическим адресом 192.168.179.129, клиент — на

Fedora с адресом 192.168.179.130. На сервере создаем интерфейс wg0 с адресом 10.10.10.1/24, на клиенте — 10.10.10.2/24. WireGuard-сервер принимает подключения по UDP-порту 51820, а для взаимной аутентификации используем пары публичного и приватного ключей. На сервере для клиента разрешён адрес 10.10.10.2/32, а на клиенте в качестве VPN-сервера указан endpoint 192.168.179.129:51820 и разрешён маршрут к 10.10.10.1/32. Подробнее см. картинку ниже.

Описание архитектуры VPN

Архитектура VPN-стенда WireGuard

Защищённый канал между Ubuntu и Fedora для доступа к внутренним ресурсам

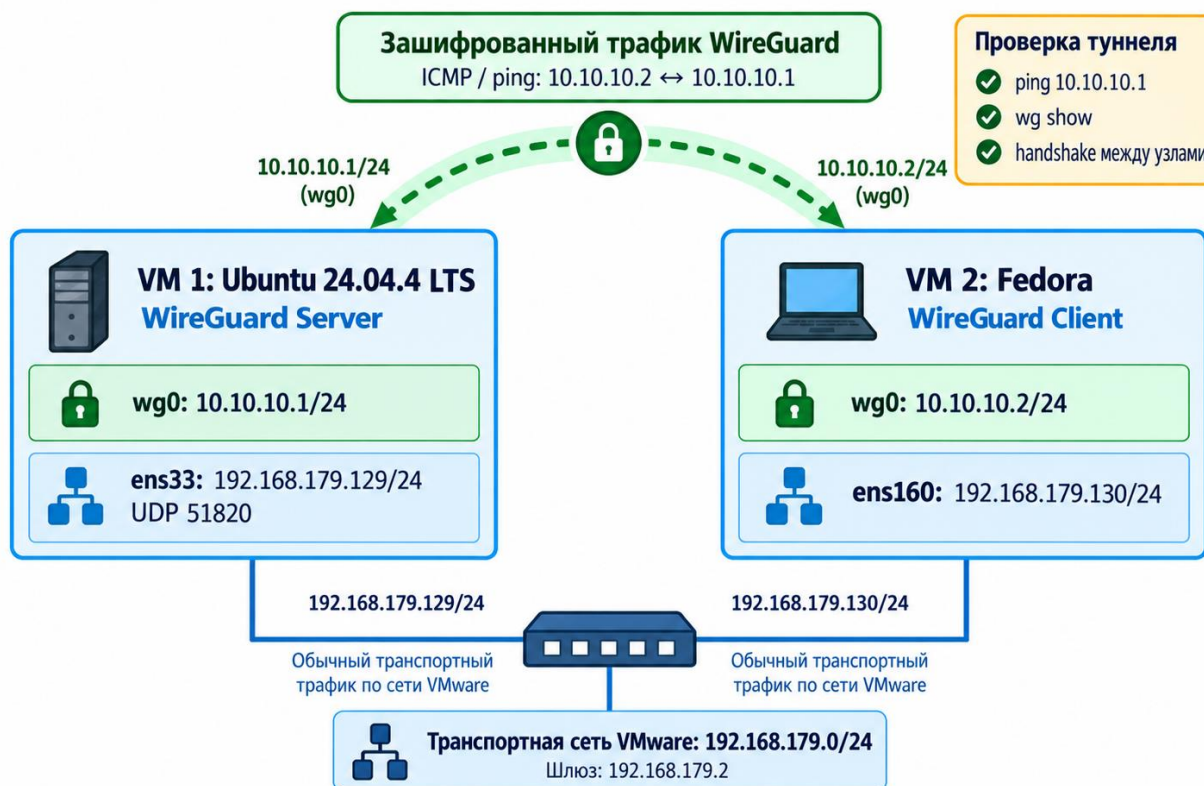


Рисунок 1. Архитектура настроенного VPN-решения

Скриншоты установки и настройки VPN

Текст полностью взят из домашней работы номер 2.

На машине с Ubuntu:

```
sudo apt update
```

```
sudo apt install wireguard -y
```

Проверка:

```
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~$ wg --version
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~$
```

На машине с Fedora:

```
sudo dnf install wireguard-tools -y
```

Проверка:

```
vladmirkuryndin@fedora:~$ sudo dnf install wireguard-tools -y
[sudo] password for vladmirkuryndin:
Updating and loading repositories:
  Fedora 42 - x86_64 - Updates                                100% | 6.4 KiB/s | 4.0 KiB | 00m01s
Repositories loaded.
Package Arch Version Repository Size
Installing:
  wireguard-tools x86_64 1.0.20210914-9.fc42 fedora 289.3 KiB

Transaction Summary:
  Installing: 1 package

Total size of inbound packages is 126 KiB. Need to download 126 KiB.
After this operation, 289 KiB extra will be used (install 289 KiB, remove 0 B).
[1/1] wireguard-tools-0:1.0.20210914-9.fc42.x86_64 100% | 139.1 KiB/s | 125.6 KiB | 00m01s
-----
[1/1] Total 100% | 108.6 KiB/s | 125.6 KiB | 00m01s
Running transaction
[1/3] Verify package files 100% | 200.0 B/s | 1.0 B | 00m00s
[2/3] Prepare transaction 100% | 2.0 B/s | 1.0 B | 00m00s
[3/3] Installing wireguard-tools-0:1.0.20210914-9.fc42.x86_64 100% | 225.4 KiB/s | 305.2 KiB | 00m01s
Complete!
vladmirkuryndin@fedora:~$ wg --version
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
vladmirkuryndin@fedora:~$
```

Настройка WireGuard

Настройка серверной части (Ubuntu)

Создаем ключи

```
sudo mkdir -p /etc/wireguard
```

```
sudo sh -c 'umask 077; wg genkey | tee /etc/wireguard/private.key | wg pubkey
> /etc/wireguard/public.key'
```

Проверяем

```
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~$ sudo cat /etc/wireguard/public.key
V0tTyz0bSlb+XwJ9PQsqL7+ikPGipXagRADbBL3zrUE=
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~$ sudo sh -c 'ls -l /etc/wireguard/*.key'
-rw----- 1 root root 45 Sep 23 11:23 /etc/wireguard/private.key
-rw----- 1 root root 45 Sep 23 11:23 /etc/wireguard/public.key
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~$
```

Создаем конфигурационный файл

```
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~$ sudo nano
/etc/wireguard/wg0.conf
```

Получаем:

```
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform: ~  
GNU nano 7.2 /etc/wireguard/wg0.conf  
[Interface]  
Address = 10.10.10.1/24  
ListenPort = 51820  
PrivateKey = IPq57Z456wK1L5uqzP8g22*0Wva0950Wf10.10.10.1/WQ=  
[Peer]  
PublicKey = beWZDxQIDpwk+nj6qbT75D0bs8WDRqA1pMNU4/3h4AY=  
AllowedIPs = 10.10.10.2/32
```

Поднимаем интерфейс

Выполняем следующие команды:

```
sudo wg-quick up wg0
```

```
ip -br a
```

```
sudo wg show
```

Получаем результат:

```
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~$ ip -br a  
lo                UNKNOWN    127.0.0.1/8 ::1/128  
ens33             UP        192.168.179.129/24 fe80::20c:29ff:fec1:84f1/64  
br-9aae4843d23f   UP        172.18.0.1/16 fe80::c0b:39ff:fea1:dc55/64  
docker0          DOWN     172.17.0.1/16  
veth9d8ab520if2   UP        fe80::105d:4bff:fe7c:57f2/64  
vetha5242390if2   UP        fe80::e450:c6ff:fe03:c5d1/64  
vethb5caa680if2   UP        fe80::44a3:9dff:fe79:1d62/64  
wg0              UNKNOWN    10.10.10.1/24  
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~$ sudo wg show  
interface: wg0  
  public key: V0tTyz0bSlb+xwJ9PQsQL7+ikPGipXagRADbBL3zrUE=  
  private key: (hidden)  
  listening port: 51820  
peer: beWZDxQIDpwk+nj6qbT75D0bs8WDRqA1pMNU4/3h4AY=  
  allowed ips: 10.10.10.2/32
```

Настройка клиентской части (Fedora)

Создаем ключи

```
sudo mkdir -p /etc/wireguard
```

```
sudo sh -c 'umask 077; wg genkey | tee /etc/wireguard/private.key | wg pubkey  
> /etc/wireguard/public.key'
```

Результат

```
vladimirkuryndin@fedora:~$ sudo mkdir -p /etc/wireguard  
sudo sh -c 'umask 077; wg genkey | tee /etc/wireguard/private.key | wg pubkey > /etc/wireguard/public.key'  
[sudo] password for vladimirkuryndin:  
vladimirkuryndin@fedora:~$ sudo cat /etc/wireguard/public.key  
beWZDxQIDpwk+nj6qbT75D0bs8WDRqA1pMNU4/3h4AY=
```

Создаем конфигурационный файл

```
sudo nano /etc/wireguard/wg0.conf
```

Результат

```
vladimirkuryndin@fedora:~ — sudo nano /etc/wireguard/wg0.conf
GNU nano 8.3 /etc/wireguard/wg0.conf
[Interface]
Address = 10.10.10.2/24
PrivateKey = blgy...
[Peer]
PublicKey = V0tTyz0bSlb+XwJ9PQsqL7+ikPGipXagRADbBL3zrUE=
Endpoint = 192.168.179.129:51820
AllowedIPs = 10.10.10.1/32
```

Поднимаем интерфейс

Выполняем следующие команды:

```
sudo wg-quick up wg0
```

```
ip -br a
```

```
sudo wg show
```

Получаем результат:

```
vladimirkuryndin@fedora:~$ sudo wg-quick up wg0
[sudo] password for vladimirkuryndin:
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.10.10.2/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
vladimirkuryndin@fedora:~$ ip -br a
lo                UNKNOWN      127.0.0.1/8 ::1/128
ens160            UP          192.168.179.130/24 fe80::218f:8a57:503d:63e9/64
wg0               UNKNOWN      10.10.10.2/24
vladimirkuryndin@fedora:~$ sudo wg show
interface: wg0
  public key: beWZDxQIDpwk+nj6qbT75D0bs8WDRqA1pMNU4/3h4AY=
  private key: (hidden)
  listening port: 36397

peer: V0tTyz0bSlb+XwJ9PQsqL7+ikPGipXagRADbBL3zrUE=
  endpoint: 192.168.179.129:51820
  allowed ips: 10.10.10.1/32
vladimirkuryndin@fedora:~$
```

Проверка

Пустим пинг с машины Fedora на Ubuntu.

```
vladimirkuryndin@fedora:~$ ping -c 4 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=1.75 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=0.705 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=0.805 ms
64 bytes from 10.10.10.1: icmp_seq=4 ttl=64 time=1.16 ms

--- 10.10.10.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3078ms
rtt min/avg/max/mdev = 0.705/1.104/1.745/0.407 ms
```

После этого проверим состояние интерфейсов на обеих виртуальных машинах.

На Fedora:

```
vladimirkuryndin@fedora:~$ sudo wg show
interface: wg0
  public key: beWZDxQIDpwk+nj6qbT75D0bs8WDRqA1pMNU4/3h4AY=
  private key: (hidden)
  listening port: 36397

peer: V0tTyz0bSlb+xwJ9PQSql7+ikPGipXagRADbBL3zrUE=
  endpoint: 192.168.179.129:51820
  allowed ips: 10.10.10.1/32
  latest handshake: 2 minutes, 31 seconds ago
  transfer: 604 B received, 692 B sent
vladimirkuryndin@fedora:~$
```

На Ubuntu:

```
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~$ sudo wg show
interface: wg0
  public key: V0tTyz0bSlb+xwJ9PQSql7+ikPGipXagRADbBL3zrUE=
  private key: (hidden)
  listening port: 51820

peer: beWZDxQIDpwk+nj6qbT75D0bs8WDRqA1pMNU4/3h4AY=
  endpoint: 192.168.179.130:36397
  allowed ips: 10.10.10.2/32
  latest handshake: 3 minutes, 51 seconds ago
  transfer: 692 B received, 604 B sent
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~$
```

После запуска интерфейса wg0 на обеих машинах проверили связность внутри VPN-туннеля командой ping 10.10.10.1 с клиентской Fedora. Обмен пакетами выполнялся успешно, без потерь, а команда wg show подтверждала установление WireGuard handshake и передачу данных между узлами.

```
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~/vpn-oauth-lab$ sudo docker ps
[sudo] password for vladimir-kuryndin:
CONTAINER ID   IMAGE                                NAMES      COMMAND                  CREATED        STATUS        PORTS
7fab4544d932   vpn-oauth-lab-web                  vpn-oauth-web  "python app.py"         25 minutes ago Up 25 minutes 10.10.10.1:5000->5000/tcp
23127efc1f6c   quay.io/keycloak/keycloak:26.7.4  vpn-oauth-keycloak  "/opt/keycloak/bin/k... 16 hours ago  Up 16 hours  8443/tcp, 9000/tcp, 10.10.10.1:8180->8080/tcp
```

Как устроено веб-приложение

Общее описание

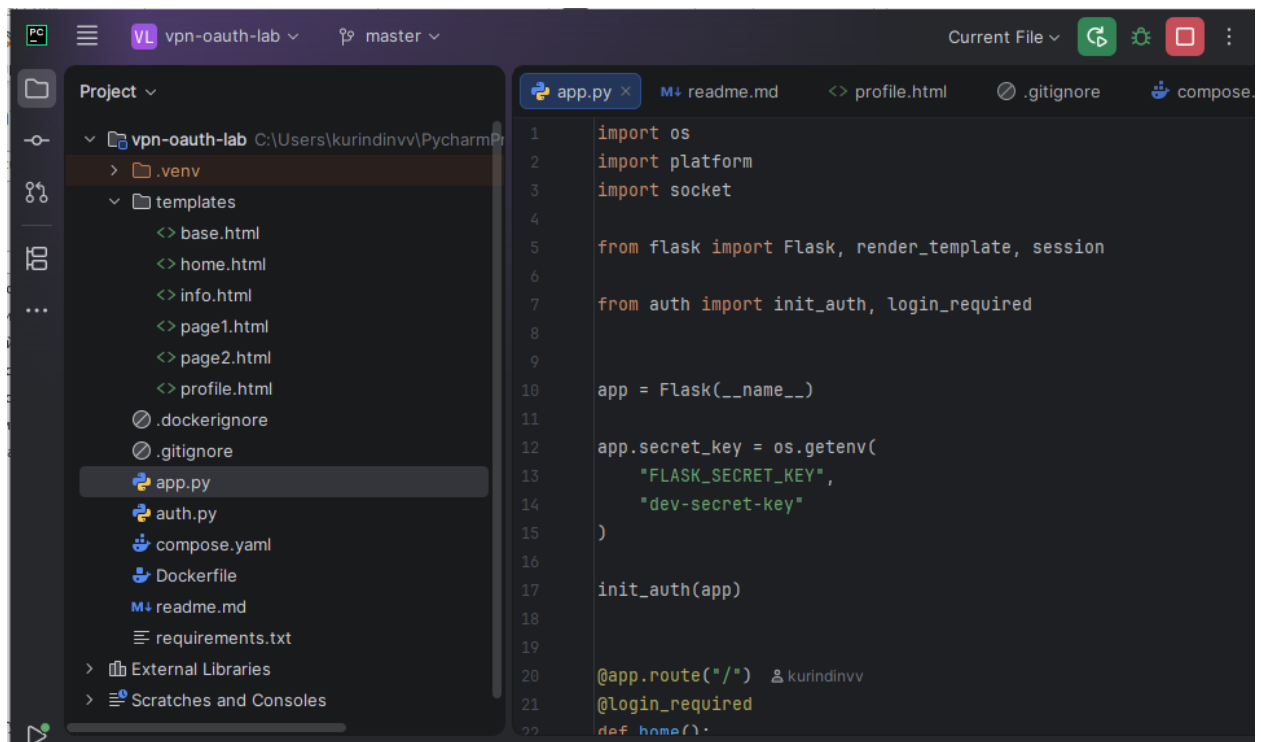
Веб-приложение реализуем на языке **Python 3.12** с использованием фреймворка **Flask 3.1.3**. Приложение имеет несколько страниц: главную страницу, две информационные страницы, страницу с информацией о сервере и страницу профиля пользователя. HTML-разметка выносим в отдельные шаблоны Jinja2 в каталоге templates, а общую структуру страниц и навигационное меню определяем в базовом шаблоне base.html.

Приложение запускаем в отдельном Docker-контейнере на серверной виртуальной машине Ubuntu. Для контейнера используем базовый образ python:3.12-slim, а управление запуском выполняется через **Docker Compose**. Веб-сервис публикуем только на VPN-адресе сервера 10.10.10.1 и порту 5000, поэтому прямой доступ к нему через обычный адрес сервера 192.168.179.129 отсутствует.

Исходный код

Исходный код веб-приложения доступен тут: [vkuryndin/vpn-oauth-lab](https://github.com/vkuryndin/vpn-oauth-lab)

Общая структура проекта



Архитектура стенда: VPN + Web Application + OAuth 2.0

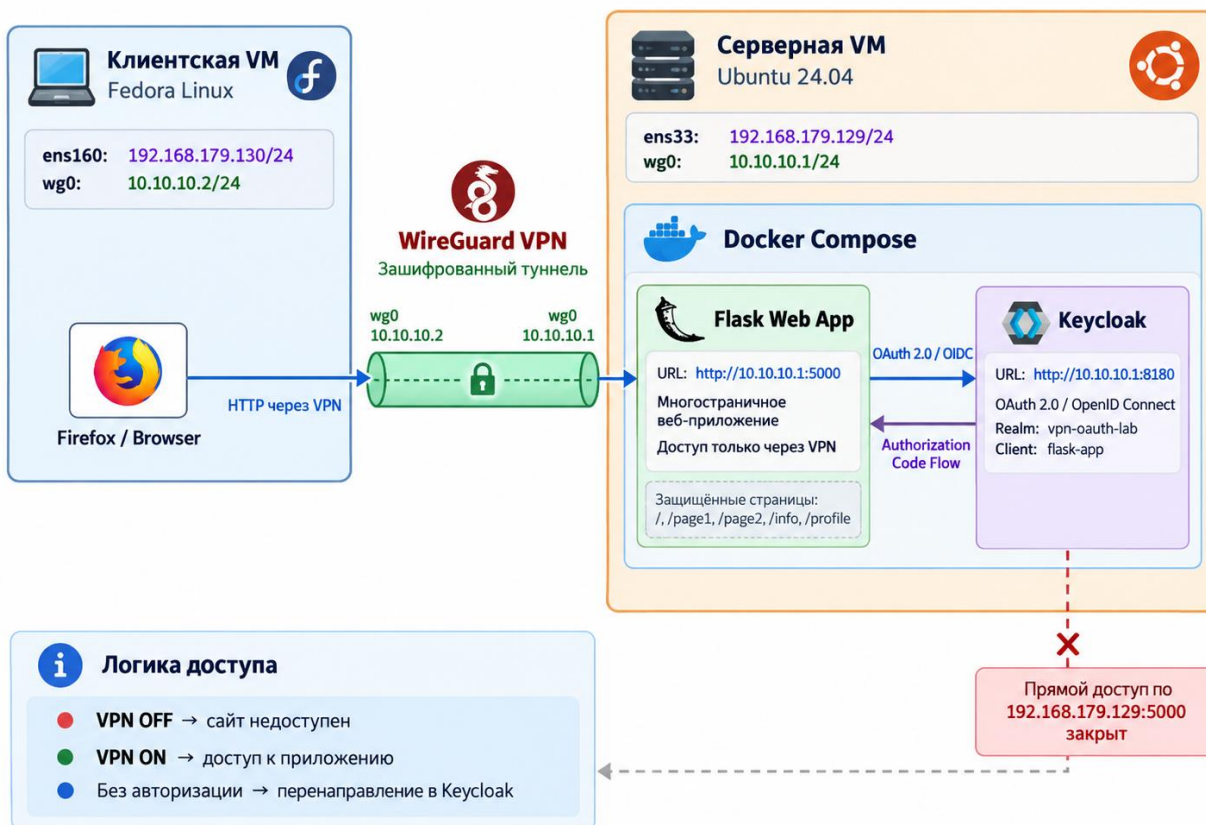


Рисунок 2. Архитектура стенда для выполнения итогового семестрового задания.

Используемое программное обеспечение

В рамках работы использовали следующее программное обеспечение:

- **WireGuard Tools 1.0.20210914** — используется для организации защищённого VPN-туннеля между клиентской и серверной виртуальными машинами.
- **Python 3.12** — язык разработки веб-приложения.
- **Flask 3.1.3** — веб-фреймворк для создания многостраничного приложения.
- **Authlib 1.8.0** — библиотека для интеграции Flask с OAuth 2.0 / OpenID Connect.
- **Requests 2.34.2** — HTTP-библиотека для взаимодействия с сервером авторизации.
- **Docker Engine 29.1.3** — используется для запуска контейнеров приложения.
- **Docker Compose 2.40.3** — используется для совместного запуска Flask-приложения и Keycloak.
- **python:3.12-slim** — базовый Docker-образ для контейнера Flask-приложения.

- **Keycloak 26.7.4** — сервер авторизации, реализующий OAuth 2.0 / OpenID Connect.

Настройка работы в докер

В `compose.yml` добавляем сервис для работы Flask. Отдельно указываем `.env` файл, в котором на сервере будем хранить секреты Keycloak и Flask.

```
services:
  web:
    build: .
    container_name: vpn-oauth-web
    env_file:
      - .env
    ports:
      - "10.10.10.1:5000:5000"
    restart: unless-stopped
```

Requirements.txt

```
Flask==3.1.3
Authlib==1.8.0
requests==2.34.2
```

Файл Docker

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

После подготовки конфигурационных файлов собираем Docker-образ и запускаем веб-приложение через Docker Compose:

```
sudo docker compose up -d --build
```

Проверяем состояние запущенного контейнера:

```
sudo docker compose ps
```

```

vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~/vpn-oauth-lab$ sudo docker compose up -d --build
WARN[0000] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 4.9s (11/11) FINISHED                                docker:default
=> [web internal] load build definition from Dockerfile          0.0s
=> => transferring dockerfile: 200B                             0.0s
=> [web internal] load metadata for docker.io/library/python:3.12-slim 0.0s
=> [web internal] load .dockerignore                             0.0s
=> => transferring context: 97B                                   0.0s
=> [web 1/5] FROM docker.io/library/python:3.12-slim           0.0s
=> [web internal] load build context                             0.0s
=> => transferring context: 3.15kB                                0.0s
=> [web 2/5] WORKDIR /app                                       0.1s
=> [web 3/5] COPY requirements.txt .                             0.0s
=> [web 4/5] RUN pip install --no-cache-dir -r requirements.txt 4.6s
=> [web 5/5] COPY . .                                           0.0s
=> [web] exporting to image                                     0.1s
=> => exporting layers                                           0.1s
=> => writing image sha256:a30578e9f3c0aa58f3ffa5864001535c6855f007c30ede5b67b61954a3769986 0.0s
=> => naming to docker.io/library/vpn-oauth-lab-web            0.0s
=> [web] resolving provenance for metadata file                 0.0s
[+] Running 3/3
✔ web Built 0.0s
✔ Network vpn-oauth-lab_default Created 0.0s
✔ Container vpn-oauth-web Started 0.2s
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~/vpn-oauth-lab$ sudo docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
vpn-oauth-web       vpn-oauth-lab-web    "python app.py"        web        12 seconds ago Up 11 seconds 10.10.10.1:5000->5000/tcp

```

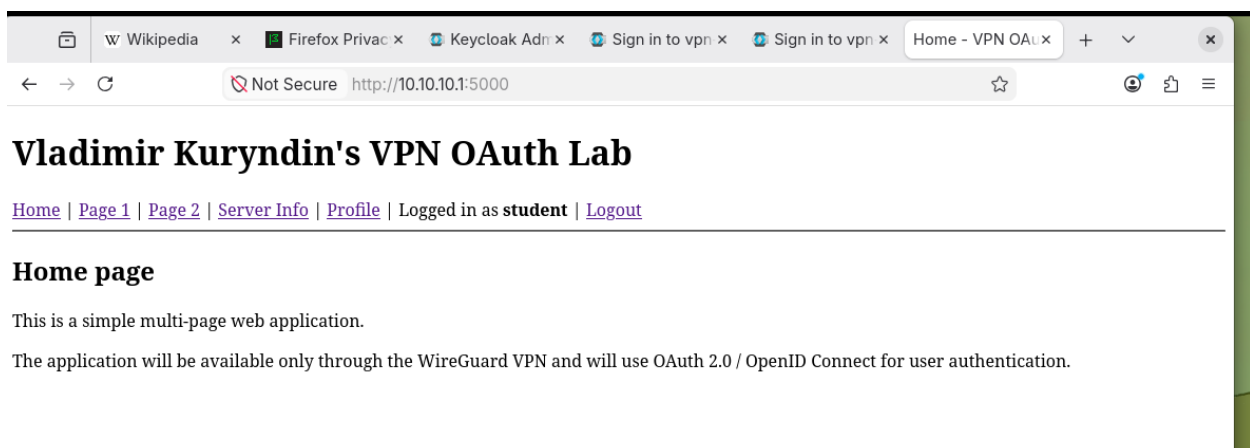
В итоге контейнер vpn-oauth-web запущен, а веб-приложение опубликовано только на VPN-адресе 10.10.10.1:5000.

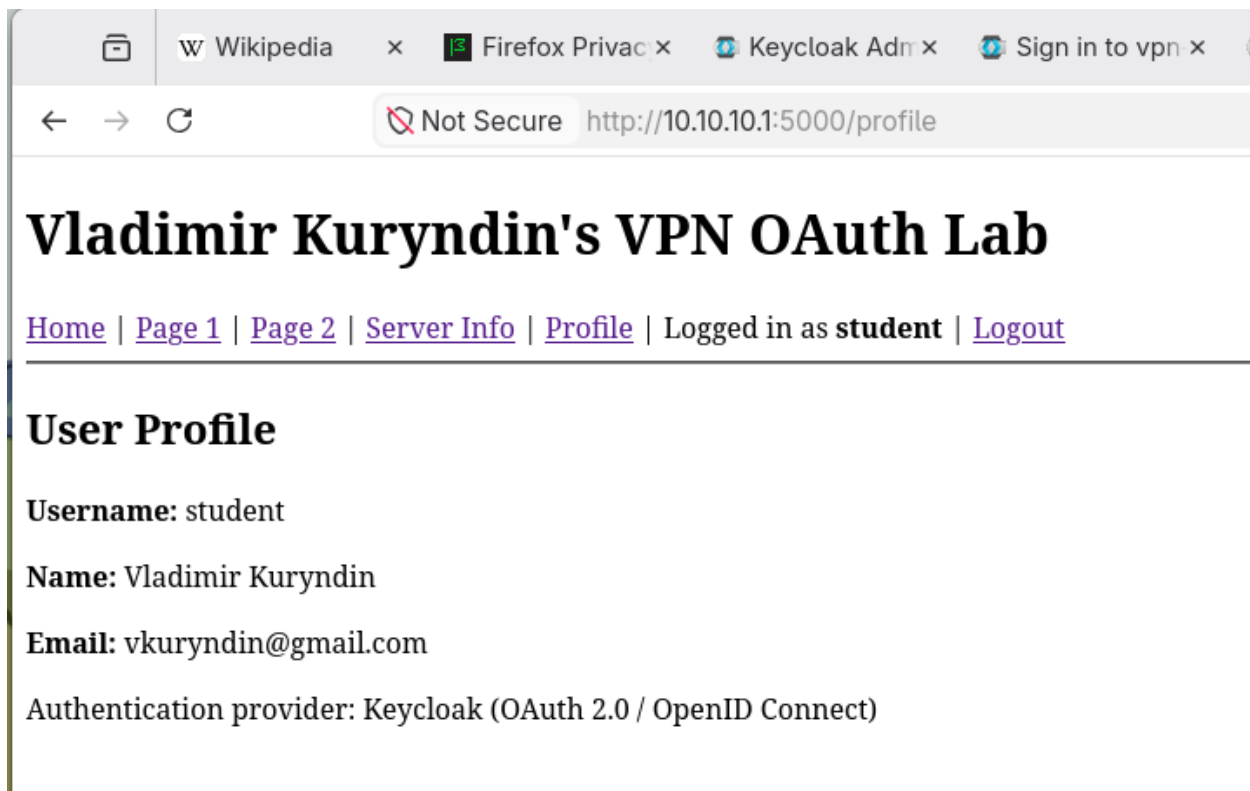
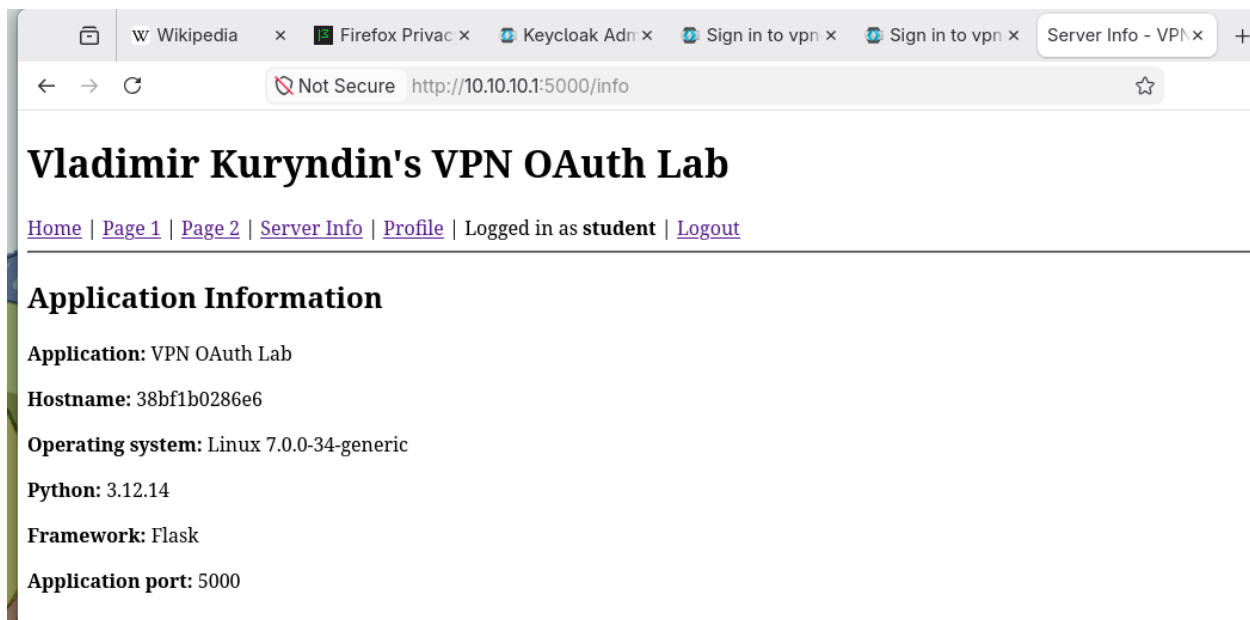
Доступ к веб-приложению только через VPN

Для выполнения требования о доступе к веб-приложению только через VPN мы опубликовали Docker-контейнер с Flask исключительно на адресе 10.10.10.1:5000. Поэтому обращение к приложению через обычный адрес сервера 192.168.179.129:5000 невозможно. При отключении WireGuard на Fedora командой `wg-quick down wg0` доступ к 10.10.10.1:5000 прекращается и возникает timeout, а после повторного запуска `wg0` веб-приложение снова становится доступно.

Скриншоты веб-приложения

Примечание. Скриншоты даны уже после настройки аутентификации и размещения сайта на целевой платформе. В адресной строке мы видим, что сайт доступен только через VPN.





Интеграция авторизации в веб-приложение

Все пользовательские страницы приложения защищаем авторизацией. Для этого используется библиотека Authlib, а сервером авторизации выступает Keycloak. **Для аутентификации пользователей используется OpenID Connect поверх OAuth 2.0 с Authorization Code Flow.** При обращении к защищённой странице неавторизованный пользователь перенаправляется на

страницу входа Keycloak. После успешной аутентификации пользователь возвращается в Flask-приложение, где создаётся сессия и становятся доступны защищённые страницы.

Настройка Keycloak

Для Keycloak настроены:

- **Realm:** vpn-oauth-lab
- **Client:** flask-app
- **Flow:** Authorization Code Flow
- **Flask URL:** http://10.10.10.1:5000
- **Keycloak URL:** http://10.10.10.1:8180

Настраиваем docker compose

В compose.yaml добавляем сервис Keycloak.

```
services:
  web:
    build: .
    container_name: vpn-oauth-web
    env_file:
      - .env
    ports:
      - "10.10.10.1:5000:5000"
    restart: unless-stopped

  keycloak:
    image: quay.io/keycloak/keycloak:26.7.4
    container_name: vpn-oauth-keycloak
    command: start-dev
    environment:
      KC_BOOTSTRAP_ADMIN_USERNAME: admin
      KC_BOOTSTRAP_ADMIN_PASSWORD: admin
    ports:
      - "10.10.10.1:8180:8080"
    mem_limit: 1g
    restart: unless-stopped
```

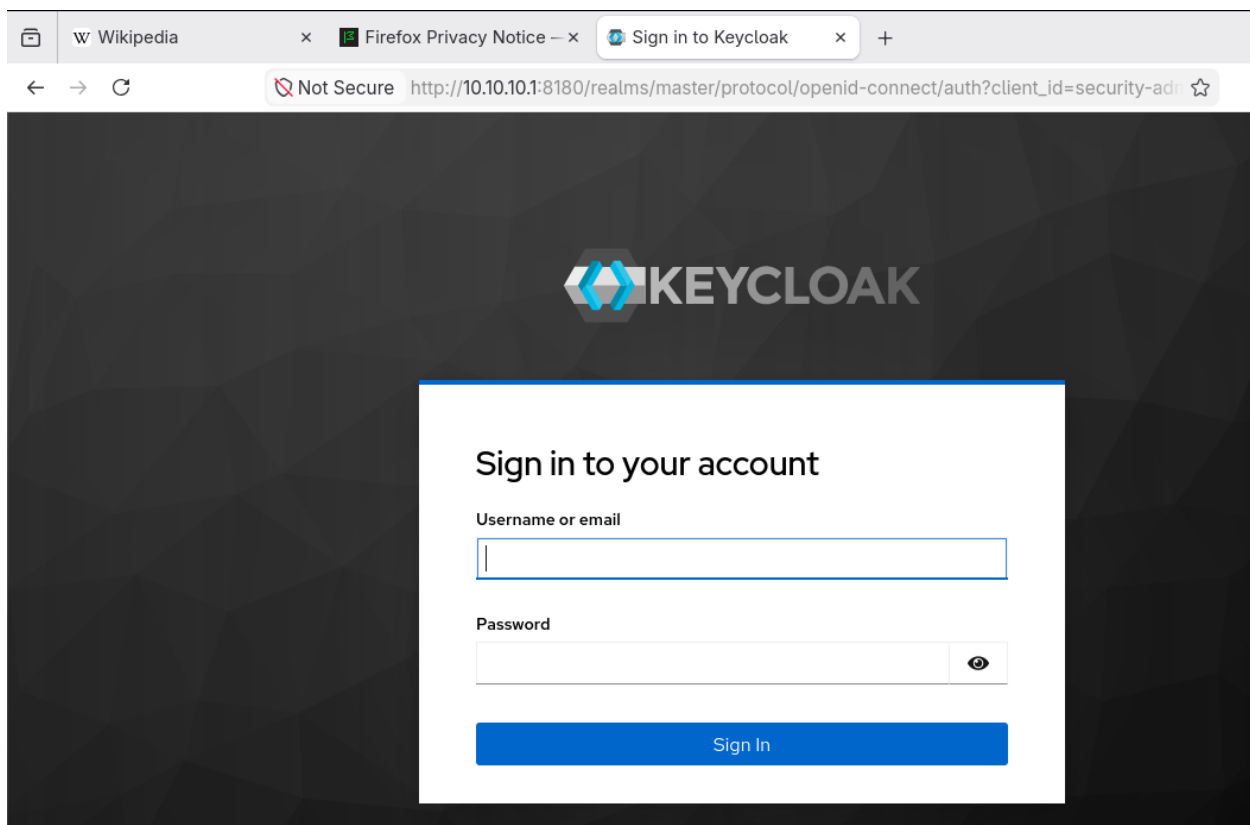
Keycloak опубликован на VPN-интерфейсе 10.10.10.1:8180.

```
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~/vpn-oauth-lab$ sudo docker compose up -d
[+] Running 5/5
  ✓ keycloak Pulled                                20.9s
    ✓ 227bdb4b4505 Pull complete                    3.0s
    ✓ bb7474a486db Pull complete                    16.2s
    ✓ b021fe485c81 Pull complete                    17.5s
    ✓ dcee140b22ee Pull complete                    17.6s
[+] Running 2/2
  ✓ Container vpn-oauth-keycloak Started            0.4s
  ✓ Container vpn-oauth-web Running                 0.0s
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~/vpn-oauth-lab$ sudo docker compose ps
```

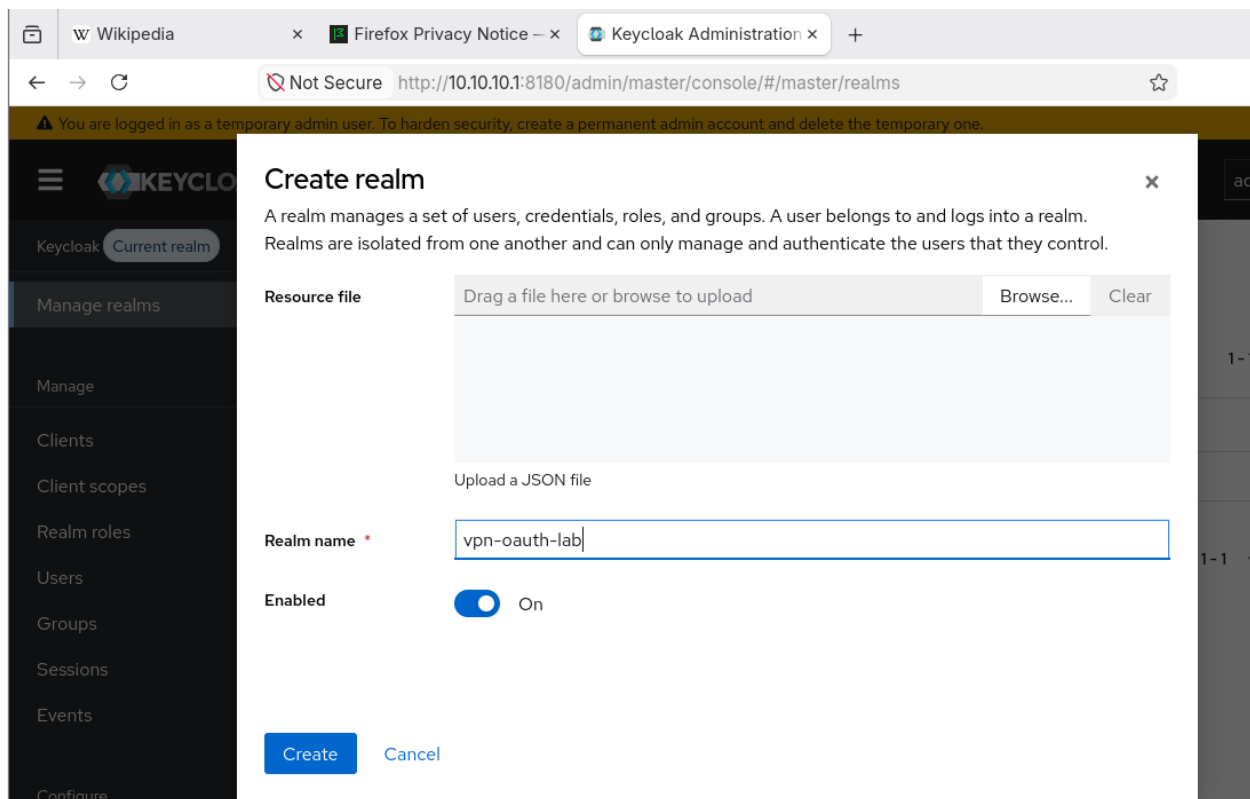
NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
vpn-oauth-keycloak	quay.io/keycloak/keycloak:26.7.4	"/opt/keycloak/bin/k..."	keycloak	8 seconds ago	Up 8 seconds
vpn-oauth-web	vpn-oauth-lab-web	"python app.py"	web	5 minutes ago	Up 5 minutes

Настройка свойств Keycloak

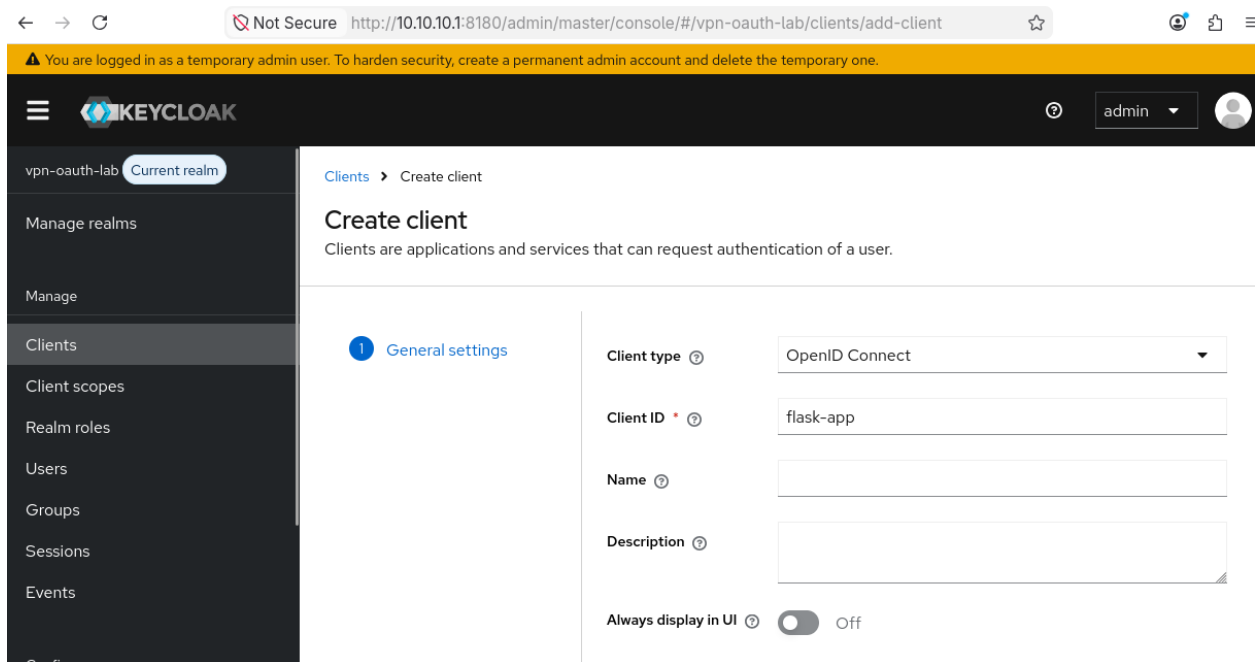
Первый вход. Используем имя пользователя admin и пароль admin. Для использования в production, конечно, надо создать полноценного администратора. Для учебных целей я решил это не менять.



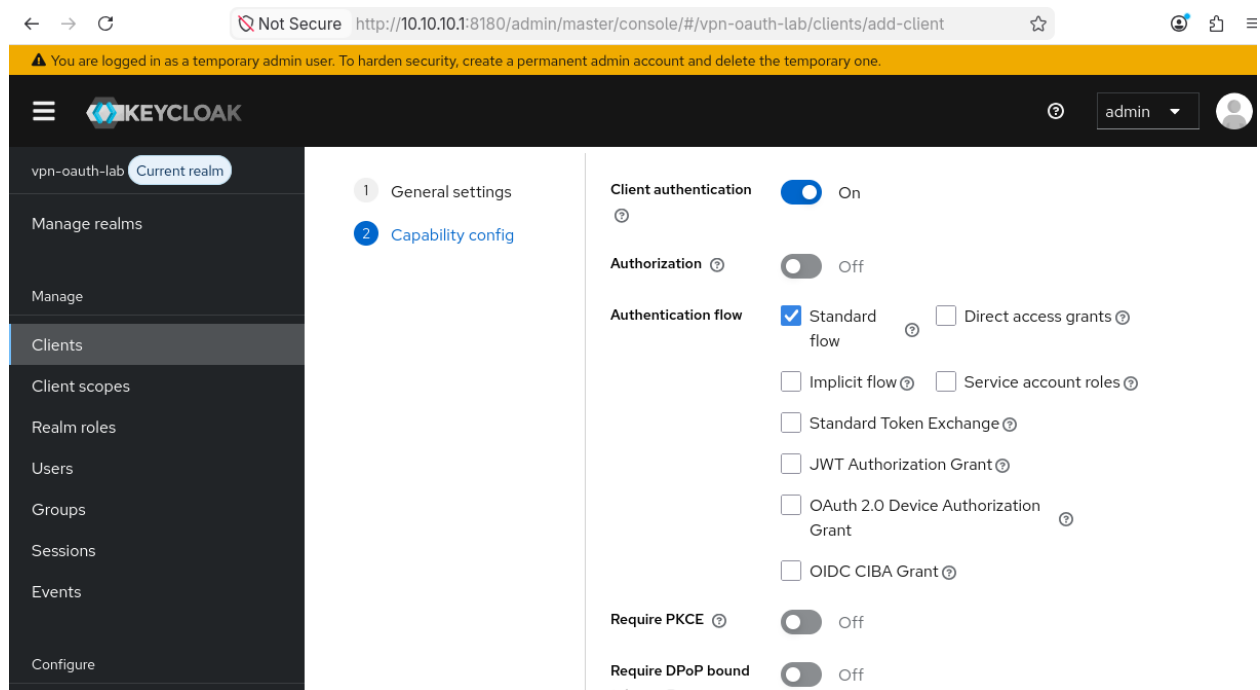
Настраиваем realm



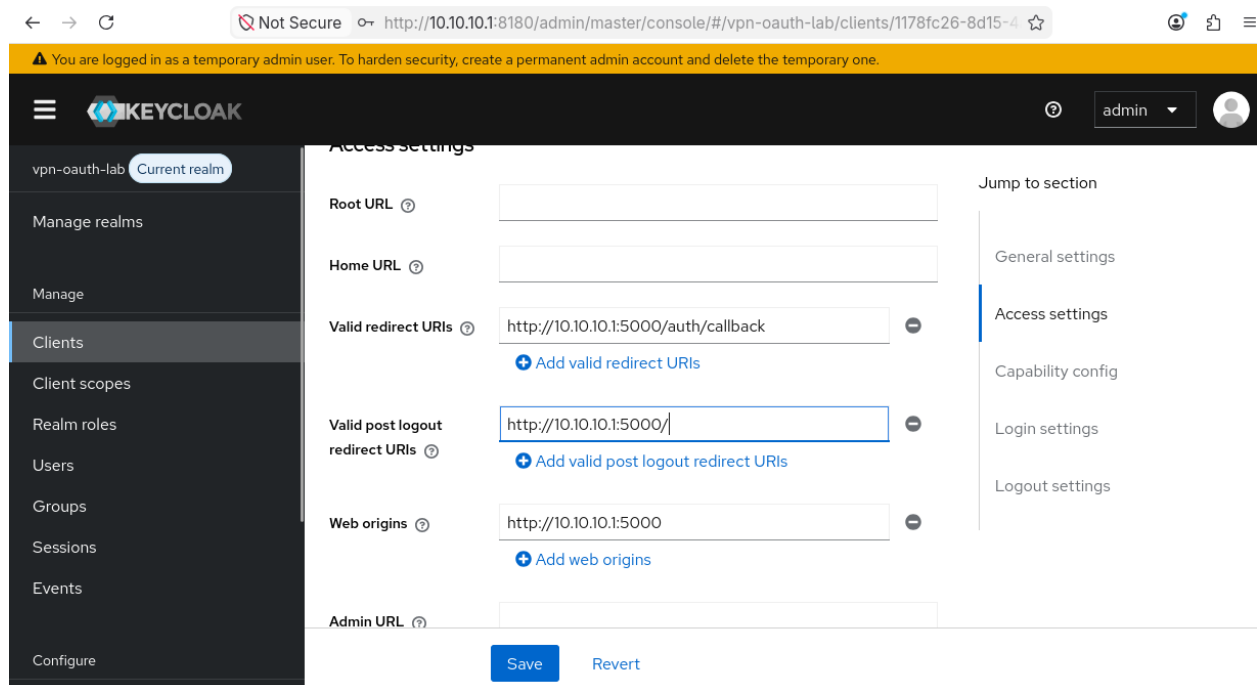
Настраиваем клиента.



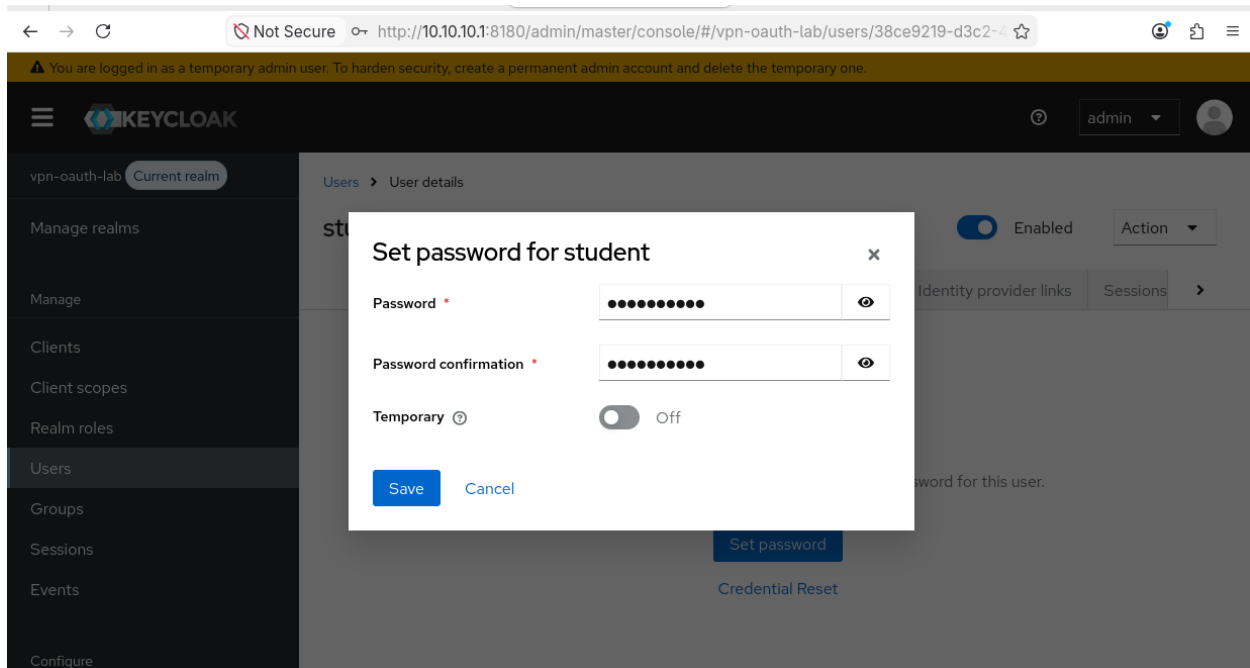
Настраиваем Client Authentication и Authentication Flow.



Настраиваем Valid redirect URIs, Valid post logout redirect URIs, Web origins.



Копируем Client secret, чтобы потом использовать его в .env файле для нашего приложения.



Создаем .env файл на Ubuntu. Там будет два секрета:

- KEYCLOAK_CLIENT_SECRET-мы скопировали при настройке клиента в Keycloak. Это секрет OAuth-клиента flask-app, нужен для взаимодействия Flask с Keycloak
- FLASK_SECRET_KEY – сгенерировали с помощью команды (.venv) PS
C:\Users\kurindinv\PycharmProjects\vpn-oauth-lab> python -c "import secrets;
print(secrets.token_hex(32))". Это внутренний секрет Flask, нужен для подписи и защиты пользовательской сессии.

Примечание. Файл .env добавлен в .gitignore, поэтому значения KEYCLOAK_CLIENT_SECRET и FLASK_SECRET_KEY не публикуются в GitHub.

```
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~/vpn-oauth-lab$ nano .env
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~/vpn-oauth-lab$ chmod 600 .env
```

Проверяем, что .env файл содержит все необходимые секреты.

```
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~/vpn-oauth-lab$ sudo docker exec vpn-oauth-web sh -c 'test -n "$KEYCLOAK_CLIENT_SECRET" && echo KEYCLOAK_CLIENT_SECRET=OK; test -n "$FLASK_SECRET_KEY" && echo FLASK_SECRET_KEY=OK'
KEYCLOAK_CLIENT_SECRET=OK
FLASK_SECRET_KEY=OK
```

Проверяем, что у нас есть все необходимые routes.

```
vladimir-kuryndin@vladimir-kuryndin-VMware-Virtual-Platform:~/vpn-oauth-lab$ sudo docker exec vpn-oauth-web flask --app app routes
* Tip: There are .env files present. Install python-dotenv to use them.
Endpoint      Methods  Rule
-----
auth_callback GET      /auth/callback
home          GET      /
info          GET      /info
login         GET      /login
logout        GET      /logout
page1         GET      /page1
page2         GET      /page2
profile       GET      /profile
static        GET      /static/<path:filename>
```

Настройка Keycloak успешно завершена.

Проверка входа с помощью аутентификации

Осуществляем вход с помощью пользователя student

Все пользовательские страницы приложения защищены аутентификацией.

Проверка, что сайт работает только через VPN

Проверяем, что VPN-канал работает.

```
vladimirkuryndin@fedora:~$ ping -c 4 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=0.811 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=0.457 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=0.859 ms
64 bytes from 10.10.10.1: icmp_seq=4 ttl=64 time=0.519 ms

--- 10.10.10.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3057ms
rtt min/avg/max/mdev = 0.457/0.661/0.859/0.175 ms
```

Проверяем, что сайт недоступен напрямую через обычный IP Ubuntu.\

```
vladimirkuryndin@fedora:~$ curl --connect-timeout 3 http://192.168.179.129:5000
curl: (7) Failed to connect to 192.168.179.129 port 5000 after 0 ms: Could not connect to server
vladimirkuryndin@fedora:~$
```

Далее отключаем VPN

```
sudo wg-quick down wg0
```

```
vladimirkuryndin@fedora:~$ sudo wg-quick down wg0
[sudo] password for vladimirkuryndin:
Sorry, try again.
[sudo] password for vladimirkuryndin:
[#] ip link delete dev wg0
vladimirkuryndin@fedora:~$ curl --connect-timeout 3 http://10.10.10.1:5000
curl: (28) Connection timed out after 3002 milliseconds
```

Теперь включаем VPN и все работает.

```
vladimirkuryndin@fedora:~$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.10.10.2/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
vladimirkuryndin@fedora:~$ curl --connect-timeout 3 http://10.10.10.1:5000
<!doctype html>
<html lang=en>
<title>Redirecting...</title>
<h1>Redirecting...</h1>
<p>You should be redirected automatically to the target URL: <a href="/login">/login</a>. If not, click the link.
vladimirkuryndin@fedora:~$
```

Проверим, что все работает, только с авторизацией

```

vladimirkuryndin@fedora:~$ for path in / /page1 /page2 /info /profile; do
  echo "=== $path ==="
  curl -s -o /dev/null -D - "http://10.10.10.1:5000$path" | grep -E 'HTTP/|Location:'
done
=== / ===
HTTP/1.1 302 FOUND
Location: /login
=== /page1 ===
HTTP/1.1 302 FOUND
Location: /login
=== /page2 ===
HTTP/1.1 302 FOUND
Location: /login
=== /info ===
HTTP/1.1 302 FOUND
Location: /login
=== /profile ===
HTTP/1.1 302 FOUND
Location: /login

```

На данном скриншоте мы видим, что после повторного включения WireGuard HTTP-доступ к приложению восстанавливается. Ответ 302 с перенаправлением на /login подтверждает, что запрос достиг Flask-приложения, после чего срабатывает механизм аутентификации.

Выводы и сложности

В результате веб-приложение было успешно развернуто в Docker, доступ к нему ограничен VPN-туннелем WireGuard, а все пользовательские страницы защищены авторизацией через Keycloak по OAuth 2.0 / OpenID Connect. Основные сложности были связаны с настройкой сетевой доступности контейнеров только через VPN-интерфейс, интеграцией Flask с Keycloak и корректной реализацией logout. Также были сложности с программированием сценария logout, чтобы он переводил пользователя на страницу login.

Улучшить решение можно за счёт перехода на HTTPS, использования production-режима Keycloak и WSGI-сервера вместо встроенного Flask development server. Секреты (KEYCLOAK_CLIENT_SECRET, FLASK_SECRET_KEY) вместо хранения в локальном .env можно вынести в **Docker Secrets** либо в специализированное хранилище, например **HashiCorp Vault**, чтобы они не хранились в открытом виде рядом с проектом и не передавались как обычные переменные окружения.